
2.7 Exercises

Reinforcement

- R-2.1 Give three examples of life-critical software applications.
- R-2.2 Give an example of a software application in which adaptability can mean the difference between a prolonged lifetime of sales and bankruptcy.
- R-2.3 Describe a component from a text-editor GUI and the methods that it encapsulates.
- R-2.4 Assume that we change the `CreditCard` class (see Code Fragment 1.5) so that instance variable `balance` has **private** visibility. Why is the following implementation of the `PredatoryCreditCard.charge` method flawed?

```
public boolean charge(double price) {  
    boolean isSuccess = super.charge(price);  
    if (!isSuccess)  
        charge(5);           // the penalty  
    return isSuccess;  
}
```

- R-2.5 Assume that we change the `CreditCard` class (see Code Fragment 1.5) so that instance variable `balance` has **private** visibility. Why is the following implementation of the `PredatoryCreditCard.charge` method flawed?

```
public boolean charge(double price) {  
    boolean isSuccess = super.charge(price);  
    if (!isSuccess)  
        super.charge(5);     // the penalty  
    return isSuccess;  
}
```

- R-2.6 Give a short fragment of Java code that uses the progression classes from Section 2.2.3 to find the eighth value of a Fibonacci progression that starts with 2 and 2 as its first two values.
- R-2.7 If we choose an increment of 128, how many calls to the `nextValue` method from the `ArithmeticProgression` class of Section 2.2.3 can we make before we cause a long-integer overflow?
- R-2.8 Can two interfaces mutually extend each other? Why or why not?
- R-2.9 What are some potential efficiency disadvantages of having very deep inheritance trees, that is, a large set of classes, A, B, C, and so on, such that B extends A, C extends B, D extends C, etc.?
- R-2.10 What are some potential efficiency disadvantages of having very shallow inheritance trees, that is, a large set of classes, A, B, C, and so on, such that all of these classes extend a single class, Z?

R-2.11 Consider the following code fragment, taken from some package:

```
public class Maryland extends State {
    Maryland() { /* null constructor */ }
    public void printMe() { System.out.println("Read it."); }
    public static void main(String[] args) {
        Region east = new State();
        State md = new Maryland();
        Object obj = new Place();
        Place usa = new Region();
        md.printMe();
        east.printMe();
        ((Place) obj).printMe();
        obj = md;
        ((Maryland) obj).printMe();
        obj = usa;
        ((Place) obj).printMe();
        usa = md;
        ((Place) usa).printMe();
    }
}

class State extends Region {
    State() { /* null constructor */ }
    public void printMe() { System.out.println("Ship it."); }
}

class Region extends Place {
    Region() { /* null constructor */ }
    public void printMe() { System.out.println("Box it."); }
}

class Place extends Object {
    Place() { /* null constructor */ }
    public void printMe() { System.out.println("Buy it."); }
}
```

What is the output from calling the main() method of the Maryland class?

R-2.12 Draw a class inheritance diagram for the following set of classes:

- Class Goat extends Object and adds an instance variable tail and methods milk() and jump().
- Class Pig extends Object and adds an instance variable nose and methods eat(food) and wallow().
- Class Horse extends Object and adds instance variables height and color, and methods run() and jump().
- Class Racer extends Horse and adds a method race().
- Class Equestrian extends Horse and adds instance variable weight and isTrained, and methods trot() and isTrained().

- R-2.13 Consider the inheritance of classes from Exercise R-2.12, and let d be an object variable of type `Horse`. If d refers to an actual object of type `Equestrian`, can it be cast to the class `Racer`? Why or why not?
- R-2.14 Give an example of a Java code fragment that performs an array reference that is possibly out of bounds, and if it is out of bounds, the program catches that exception and prints the following error message:
“Don’t try buffer overflow attacks in Java!”
- R-2.15 If the parameter to the `makePayment` method of the `CreditCard` class (see Code Fragment 1.5) were a negative number, that would have the effect of *raising* the balance on the account. Revise the implementation so that it throws an `IllegalArgumentException` if a negative amount is sent as a parameter.

Creativity

- C-2.16 Suppose you are on the design team for a new e-book reader. What are the primary classes and methods that the Java software for your reader will need? You should include an inheritance diagram for this code, but you don’t need to write any actual code. Your software architecture should at least include ways for customers to buy new books, view their list of purchased books, and read their purchased books.
- C-2.17 Most modern Java compilers have optimizers that can detect simple cases when it is logically impossible for certain statements in a program to ever be executed. In such cases, the compiler warns the programmer about the useless code. Write a short Java method that contains code for which it is provably impossible for that code to ever be executed, yet the Java compiler does not detect this fact.
- C-2.18 The `PredatoryCreditCard` class provides a `processMonth()` method that models the completion of a monthly cycle. Modify the class so that once a customer has made ten calls to `charge` during a month, each additional call to that method in the current month results in an additional \$1 surcharge.
- C-2.19 Modify the `PredatoryCreditCard` class so that a customer is assigned a minimum monthly payment, as a percentage of the balance, and so that a late fee is assessed if the customer does not subsequently pay that minimum amount before the next monthly cycle.
- C-2.20 Assume that we change the `CreditCard` class (see Code Fragment 1.5) so that instance variable `balance` has **private** visibility, but a new **protected** method is added, with signature `setBalance(newBalance)`. Show how to properly implement the method `PredatoryCreditCard.processMonth()` in this setting.
- C-2.21 Write a program that consists of three classes, A , B , and C , such that B extends A and that C extends B . Each class should define an instance variable named “ x ” (that is, each has its own variable named x). Describe a way for a method in C to access and set A ’s version of x to a given value, without changing B or C ’s version.

- C-2.22 Explain why the Java dynamic dispatch algorithm, which looks for the method to invoke for a call `obj.foo()`, will never get into an infinite loop.
- C-2.23 Modify the `advance` method of the `FibonacciProgression` class so as to avoid use of any temporary variable.
- C-2.24 Write a Java class that extends the `Progression` class so that each value in the progression is the absolute value of the difference between the previous two values. You should include a default constructor that starts with 2 and 200 as the first two values and a parametric constructor that starts with a specified pair of numbers as the first two values.
- C-2.25 Redesign the `Progression` class to be abstract and generic, producing a sequence of values of generic type `T`, and supporting a single constructor that accepts an initial value. Make all corresponding modifications to the rest of the classes in our hierarchy so that they remain as nongeneric classes, while inheriting from the new generic `Progression` class.
- C-2.26 Use a solution to Exercise C-2.25 to create a new progression class for which each value is the square root of the previous value, represented as a `Double`. You should include a default constructor that has 65,536 as the first value and a parametric constructor that starts with a specified number as the first value.
- C-2.27 Use a solution to Exercise C-2.25 to reimplement the `FibonacciProgression` subclass to rely on the `BigInteger` class, in order to avoid overflows all together.
- C-2.28 Write a set of Java classes that can simulate an Internet application in which one party, Alice, is periodically creating a set of packets that she wants to send to Bob. An Internet process is continually checking if Alice has any packets to send, and if so, it delivers them to Bob's computer; Bob is periodically checking if his computer has a packet from Alice, and if so, he reads and deletes it.
- C-2.29 Write a Java program that inputs a polynomial in standard algebraic notation and outputs the first derivative of that polynomial.

Projects

- P-2.30 Write a Java program that inputs a document and then outputs a bar-chart plot of the frequencies of each alphabet character that appears within that document.
- P-2.31 Write a Java program to simulate an ecosystem containing two types of creatures, *bears* and *fish*. The ecosystem consists of a river, which is modeled as a relatively large array. Each cell of the array should contain an `Animal` object, which can be a `Bear` object, a `Fish` object, or **null**. In each time step, based on a random process, each animal either attempts to move into an adjacent array cell or stay where it is. If two animals of the same type are about to collide in the same cell, then they stay where they are, but they create a new instance of that type of animal, which is placed in a random empty (i.e., previously **null**) cell in the array. If a bear and a fish collide, however, then the fish dies (i.e., it disappears). Use actual object creation, via the **new** operator, to model the creation of new objects, and provide a visualization of the array after each time step.

- P-2.32 Write a simulator as in the previous project, but add a boolean gender field and a floating-point strength field to each Animal object. Now, if two animals of the same type try to collide, then they only create a new instance of that type of animal if they are of different genders. Otherwise, if two animals of the same type and gender try to collide, then only the one of larger strength survives.
- P-2.33 Write a Java program that simulates a system that supports the functions of an e-book reader. You should include methods for users of your system to “buy” new books, view their list of purchased books, and read their purchased books. Your system should use actual books, which have expired copyrights and are available on the Internet, to populate your set of available books for users of your system to “purchase” and read.
- P-2.34 Define a Polygon interface that has methods `area()` and `perimeter()`. Then implement classes for Triangle, Quadrilateral, Pentagon, Hexagon, and Octagon, which implement this interface, with the obvious meanings for the `area()` and `perimeter()` methods. Also implement classes, `IsoscelesTriangle`, `EquilateralTriangle`, `Rectangle`, and `Square`, which have the appropriate inheritance relationships. Finally, write a simple user interface, which allows users to create polygons of the various types, input their geometric dimensions, and then output their area and perimeter. For extra effort, allow users to input polygons by specifying their vertex coordinates and be able to test if two such polygons are similar.
- P-2.35 Write a Java program that inputs a list of words, separated by whitespace, and outputs how many times each word appears in the list. You need not worry about efficiency at this point, however, as this topic is something that will be addressed later in this book.
- P-2.36 Write a Java program that can “make change.” Your program should take two numbers as input, one that is a monetary amount charged and the other that is a monetary amount given. It should then return the number of each kind of bill and coin to give back as change for the difference between the amount given and the amount charged. The values assigned to the bills and coins can be based on the monetary system of any current or former government. Try to design your program so that it returns the fewest number of bills and coins as possible.

Chapter Notes

For a broad overview of developments in computer science and engineering, we refer the reader to *The Computer Science and Engineering Handbook* [89]. For more information about the Therac-25 incident, please see the paper by Leveson and Turner [65].

The reader interested in studying object-oriented programming further is referred to the books by Booch [16], Budd [19], and Liskov and Guttag [67]. Liskov and Guttag also provide a nice discussion of abstract data types, as does the book chapter by Demurjian [28] in the *The Computer Science and Engineering Handbook* [89]. Design patterns are described in the book by Gamma *et al.* [37].

